SurveySmart — Test Suite & Results

New automated test suite covering the fixes and core logic from the earlier code review, run end-to-end.
Project: D:\Claude_workspace\surveysmart · Date: 2026-07-20

36 / 36 tests passing	2 suites (backend + frontend)	0 extra packages installed

Backend: 29/29 passing across 5 suites. Frontend: 7/7 passing across 2 suites. Both run with Node's built-in test runner (`node --test`) — no Jest, Vitest, or Supertest install was needed (see note below on why).

Why Node's built-in test runner

`npm install` for Jest/Supertest (backend) and Vitest (frontend) repeatedly timed out or left node_modules in a corrupted, half-installed state on this machine's mounted drive. Rather than keep fighting a slow package registry, the suite was built entirely on Node 22's built-in `node:test` + `node:assert` — zero install, and it already ships with the Node version this project targets. The trade-off: no JSDOM/component-rendering tests for Vue components (see “Not covered” below).

Backend suites (backend/tests/) — 29 tests, all passing

File	Covers	#	Result
middleware.test.js	auth.js (JWT verify), isAdmin.js, isHeadAdmin.js — missing/invalid/valid tokens, role gating for both middlewares	7	✓ pass
googleAuthService.test.js	OAuth state-to-user scoping fix — tokens only returned to the user who registered the state, cross-user access blocked, single-use CSRF state, removeTokens cleanup	6	✓ pass
publicResponsesWebhook.test.js	New webhook shared-secret middleware — skipped when unset, rejects missing/wrong secret, accepts header or query param	5	✓ pass
responseController.test.js	chartData() — scale min:0 preserved (?? fix regression test), default 1-5 fallback; submit() — required-question validation, active-survey-only guard, happy path	5	✓ pass
authController.test.js	register field/length validation; login — identical generic message for unknown user vs. wrong password (no enumeration), suspended-account block, successful login never leaks the password hash	6	✓ pass

Frontend suites (frontend/tests/) — 7 tests, all passing

File	Covers	#	Result
useSurveyStatus.test.mjs	New composable (formatDate, badgeClass, badgeText, interpClass, interpText) extracted during the earlier refactor — status mapping and score-band boundaries	4	✓ pass
scaleMinMax.test.mjs	SurveyFillView.vue scaleMin/scaleMax — extracts and executes the real shipped function source (not a reimplementation) to confirm min:0 is preserved (?? fix) and defaults still apply	3	✓ pass

---

Extra fixes made while writing tests

Writing the OAuth-state test surfaced two small issues in the code under test, fixed along the way:

- **googleAuth.js loaded the googleapis package at module top-level** — that import alone took 25+ seconds on this machine, which meant just requiring the service for a unit test would hang. Moved the `require('googleapis')` inside `getClient()`, the only function that actually needs it — the state-scoping logic (registerPendingState/getTokens/etc.) never needed the package at all, so this also makes every non-Google-API-calling code path load faster.
- **The 10-minute pending-state cleanup timer wasn't unref'd** — a lingering `setTimeout` kept the Node process alive, which would delay graceful shutdown and made test processes hang until the timer fired. Added `.unref()` so it can't keep the process open.

How to run the tests

`cd backend && npm test` (runs tests/*.test.js via node --test)
`cd frontend && npm test` (runs tests/*.test.mjs via node --test)
Both require Node 18+ (this machine has Node 22); no other setup or install is needed.

Not covered by this suite

This is a fast, dependency-free unit-test layer over the logic most likely to regress — it is not a full test pyramid. Left out, and worth adding if the project wants deeper coverage:

- **Real database integration tests** — everything here mocks db.query/getConnection; nothing runs against an actual MySQL instance (surveyController CRUD, admin endpoints, share/permission joins).
- **Full HTTP-level route tests** — routes are exercised by calling controller functions directly with fake req/res, not by sending real HTTP requests through Express + middleware chains end-to-end.
- **Google Forms sync logic** (googleFormsSync.js, formSyncPoller.js) — untested; would need a mocked Google Forms API client.
- **Vue component rendering/interaction tests** — no JSDOM or component-mount testing (would need Vitest or a similar bundler-aware test runner); only framework-free logic was tested on the frontend.
- **Browser/E2E flows** — login, survey creation, filling out a public survey, Google Forms OAuth popup flow, etc. are not exercised end-to-end.